

Chapitre 2 : Les fichiers, garder ses données

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Fil rouge** : le registre des colis de Yonnee Express · **Version** : 2026-10-23

Fiche du chapitre

Objectifs	A la fin du chapitre, l'apprenant sait : 1) écrire, lire et compléter un fichier texte avec `open` et `with` ; 2) choisir l'encodage et expliquer une `UnicodeDecodeError` ; 3) manipuler des chemins et des dossiers avec `pathlib` ; 4) écrire et lire un fichier CSV avec le module `csv` ; 5) lire un fichier abîmé sans planter, en signalant les lignes invalides ; 6) enregistrer et relire des données en JSON ; 7) intercepter les erreurs de fichiers.
Prérequis	Chapitre 1 (exceptions, `try` / `except`, `raise`)
Durée	🟢 Débutant : 8 h · 🔵 Standard : 6 h · 🟣 Expert : 4 h (atelier compris)
Matériel	Python 3.12 ou plus récent (vérifié avec Python 3.13), VS Code, le fichier du registre d'octobre (fourni)
Livrable	L'atelier 2 : le registre chargé, nettoyé et réenregistré, publié sur GitHub

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Créez un dossier `essais-ch2`, ouvrez-le dans VS Code, et lancez chaque fichier dans le terminal **depuis ce dossier** (`python essai1.py`, ou `py essai1.py` sous Windows) : les fichiers créés par les exemples apparaîtront à côté.

---

1. Pourquoi des fichiers ?

A la fin de l'atelier 1, le programme de saisie de Yonnee Express fonctionnait sans planter. Mais quand on le ferme, **tout disparaît** : les livraisons ne vivaient que dans la mémoire de l'ordinateur, le temps de l'exécution. Pour garder des données d'une fois sur l'autre, les partager ou les analyser, il faut les écrire dans un **fichier**.

Format	A quoi il ressemble	Quand l'utiliser
Texte (`.txt`)	Des lignes de texte libre	Un journal, des notes, un rapport
CSV (`.csv`)	Un tableau : une ligne par enregistrement, des colonnes séparées par `;` ou `,`	Des données en tableau, qu'on veut aussi ouvrir dans Excel
JSON (`.json`)	Des dictionnaires et des listes, écrits comme en Python	Une configuration, des données imbriquées, un échange avec un site web

2. Les fichiers texte

2.1 Écrire

Créez `essai1.py` :

```
with open("journal.txt", "w", encoding="utf-8") as fichier:
    fichier.write("Livraison à Médina : 1 000 F\n")
    fichier.write("Livraison à Pikine : 2 500 F\n")
print("Fichier écrit")
```

```
Fichier écrit
```

Un fichier `journal.txt` est apparu dans le dossier. Ouvrez-le dans VS Code : il contient les deux lignes.

Morceau	Ce qu'il veut dire
`open("journal.txt", "w", ...)`	Ouvre (ou crée) le fichier `journal.txt` en mode **écriture** (`w` , *write*). Attention : si le fichier existe déjà, son contenu est **effacé**
`encoding="utf-8"`	L'**encodage** : la façon d'écrire les lettres en octets. UTF-8 sait écrire toutes les lettres (é, à, ñ, ë du wolof...). Mettez-le **toujours** (section 3)
`with ... as fichier:`	Ouvre le fichier, le range dans la variable `fichier` , et le **referme automatiquement** à la fin du bloc décalé, même si une exception survient
`fichier.write(texte)`	Écrit le texte. Elle n'ajoute **pas** de retour à la ligne : c'est le `\n` à la fin qui le fait

> 🟢 **Débutant : pourquoi `with` ?** Un fichier ouvert doit être **refermé** : sinon, ce qui a été écrit peut rester en mémoire et ne jamais arriver sur le disque, et le fichier reste bloqué pour les autres programmes (Excel, par exemple). Sans `with`, il faudrait écrire `fichier.close()` à la fin, et ne jamais l'oublier, même en cas d'erreur. `with` le fait pour vous : utilisez-le toujours.

2.2 Lire

Créez `essai2.py` :

```
with open("journal.txt", encoding="utf-8") as fichier:
    contenu = fichier.read()
print(repr(contenu))

with open("journal.txt", encoding="utf-8") as fichier:
    for numero, ligne in enumerate(fichier, start=1):
        print(numero, ligne.strip())
```

```
'Livraison à Médina : 1 000 F\nLivraison à Pikine : 2 500 F\n'
1 Livraison à Médina : 1 000 F
2 Livraison à Pikine : 2 500 F
```

Morceau	Ce qu'il veut dire
`open("journal.txt", encoding="utf-8")`	Sans mode, le fichier est ouvert en **lecture** (`"r"` , *read*)
`fichier.read()`	Tout le contenu, en **un seul** texte
`repr(contenu)`	Affiche le texte tel que Python le voit, avec les `\n` visibles
`for ligne in fichier:`	Parcourt le fichier **ligne par ligne**, sans tout charger en mémoire (pratique pour les gros fichiers)
`enumerate(..., start=1)`	Donne en plus le numéro de chaque ligne, à partir de 1
`ligne.strip()`	Retire les espaces et le `\n` au début et à la fin de la ligne

2.3 Ajouter à la fin

Créez `essai3.py` :

```
with open("journal.txt", "a", encoding="utf-8") as fichier:
    fichier.write("Livraison à Yoff : 2 000 F\n")

with open("journal.txt", encoding="utf-8") as fichier:
    lignes = fichier.readlines()
print(len(lignes), "lignes")
print(lignes[-1].strip())
```

```
3 lignes
Livraison à Yoff : 2 000 F
```

Mode	Effet	Si le fichier n'existe pas
`"r"` (par défaut)	Lire	Erreur `FileNotFoundError`
`"w"`	Écrire, en **effaçant** l'ancien contenu	Il est créé
`"a"`	Ajouter à la **fin** (*append*)	Il est créé
`"x"`	Créer un **nouveau** fichier	Il est créé ; s'il existe, erreur `FileExistsError`

`readlines()` renvoie la liste de toutes les lignes (chacune avec son `\n`).

> 🖍️ **Essayez** : relancez `essai1.py`, puis `essai3.py`. Combien de lignes ? `essai1.py` a tout effacé avec `"w"`, et `essai3.py` a ajouté une ligne : 3 lignes. Relancez `essai3.py` seul trois fois : 4, 5, 6 lignes. Le mode `"a"` ne supprime jamais rien.

3. L'encodage : le piège des accents

Un fichier n'est qu'une suite d'**octets**. L'encodage dit comment transformer les lettres en octets, et inversement. Si on écrit avec un encodage et qu'on relit avec un autre, les accents sont abîmés ou la lecture échoue. Créez `essai4.py` :

```python
with open("ancien.txt", "w", encoding="cp1252") as fichier:
    fichier.write("Livraison à Médina\n")

with open("ancien.txt", encoding="utf-8") as fichier:
    print(fichier.read())
```

```
Traceback (most recent call last):
  File "...\essai4.py", line 5, in <module>
    print(fichier.read())
          ~~~~~~~~~~~~^^
  File "<frozen codecs>", line 325, in decode
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe0 in position 10: invalid continuation byte
```

Traduction : « le décodeur UTF-8 ne peut pas lire l'octet 0xe0 à la position 10 ». L'octet `0xe0` est le « à » écrit en **cp1252**, l'ancien encodage de Windows ; en UTF-8, le « à » s'écrit avec deux octets.

Encodage	Où on le rencontre
UTF-8	Le standard actuel : sites web, Linux, macOS, VS Code, la plupart des fichiers modernes
cp1252 (ou « Windows-1252 », proche de « latin-1 »)	Les anciens fichiers de Windows, certains exports d'Excel ou de vieux logiciels

Deux règles :

1. **Écrivez toujours avec `encoding="utf-8"`.**
2. Si un fichier venu d'ailleurs donne une `UnicodeDecodeError`, il a sans doute été écrit en cp1252 : relisez-le avec `encoding="cp1252"`.

> 🔵 **Standard : et sans `encoding` ?** Sans `encoding`, Python utilise l'encodage par défaut du système. Sous Windows, c'est encore **cp1252** (`locale.getpreferredencoding()` affiche `cp1252`) ; sous Linux et macOS, c'est UTF-8. Un programme sans `encoding` peut donc fonctionner chez vous et abîmer les accents chez un camarade. Précisez-le toujours.

4. Les chemins et les dossiers avec `pathlib`

Jusqu'ici, les fichiers étaient créés **dans le dossier courant** (celui depuis lequel vous lancez `python`). Pour ranger ses fichiers dans des dossiers, et écrire des chemins qui marchent sous Windows comme sous Linux, on utilise `pathlib`. Créez `essai5.py` :

```
from pathlib import Path

dossier = Path("registres")
dossier.mkdir(exist_ok=True)
chemin = dossier / "2026-10-23.txt"
chemin.write_text("Médina;1000\nPikine;2500\n", encoding="utf-8")

print(chemin)
print(chemin.name, chemin.stem, chemin.suffix)
print(chemin.exists(), (dossier / "absent.txt").exists())
print(chemin.read_text(encoding="utf-8").splitlines())
for fichier in sorted(dossier.glob("*.txt")):
    print("Trouvé :", fichier.name)
```

```
registres\2026-10-23.txt
2026-10-23.txt 2026-10-23 .txt
True False
['Médina;1000', 'Pikine;2500']
Trouvé : 2026-10-23.txt
```

Morceau	Ce qu'il veut dire
`Path("registres")`	Un **chemin** (ici, un dossier relatif au dossier courant)
`dossier.mkdir(exist_ok=True)`	Crée le dossier ; `exist_ok=True` évite une erreur s'il existe déjà
`dossier / "2026-10-23.txt"`	**Assemble** un chemin avec `/`, quel que soit le système (Windows affiche `\`, Linux `/`)
`chemin.write_text(...)` / `read_text(...)`	Écrit ou lit tout le fichier en une ligne (pratique pour les petits fichiers)

Morceau	Ce qu'il veut dire
`.name` , `.stem` , `.suffix`	Le nom complet, le nom sans l'extension, l'extension
`.exists()`	`True` si le fichier ou le dossier existe
`dossier.glob("*.txt")`	Tous les fichiers du dossier dont le nom finit par `.txt`
`.splitlines()`	Découpe un texte en liste de lignes, sans les `\n`

> 🟢 **Débutant : chemin relatif ou absolu ?** `Path("registres")` est **relatif** : il dépend du dossier depuis lequel on lance le programme. Si vous lancez `python essais-ch2/essai5.py` depuis le dossier parent, le dossier `registres` sera créé dans le dossier parent ! Lancez toujours vos programmes depuis leur propre dossier (dans VS Code : *Terminal* > *Nouveau terminal*, puis `cd` vers le dossier). Un chemin **absolu** commence à la racine : `C:\Users\...` sous Windows, `/home/...` sous Linux.

5. Les fichiers CSV

5.1 Écrire un tableau

Le **CSV** (*comma-separated values*) est le format des tableaux : une ligne d'en-tête avec les noms des colonnes, puis une ligne par enregistrement. En France et au Sénégal, Excel attend un **point-virgule** comme séparateur (la virgule sert aux décimales). Créez `essai6.py` :

```
import csv

livraisons = [
    {"date": "2026-10-20", "quartier": "Médina", "poids": 2.5, "montant": 15000, "frais": 1000},
    {"date": "2026-10-20", "quartier": "Pikine", "poids": 7.0, "montant": 42000, "frais": 3000},
    {"date": "2026-10-21", "quartier": "Yoff", "poids": 1.2, "montant": 8000, "frais": 2000},
]
colonnes = ["date", "quartier", "poids", "montant", "frais"]
with open("registre.csv", "w", newline="", encoding="utf-8") as fichier:
    ecrivain = csv.DictWriter(fichier, fieldnames=colonnes, delimiter=";")
    ecrivain.writeheader()
    ecrivain.writerows(livraisons)

with open("registre.csv", encoding="utf-8") as fichier:
    print(fichier.read())
```

```
date;quartier;poids;montant;frais
2026-10-20;Médina;2.5;15000;1000
2026-10-20;Pikine;7.0;42000;3000
2026-10-21;Yoff;1.2;8000;2000
```

Morceau	Ce qu'il veut dire
`import csv`	Le module de Python pour les fichiers CSV (pas d'installation)

Morceau	Ce qu'il veut dire
`newline=""`	Laisse le module `csv` gérer lui-même les fins de ligne (voir l'encadré)
`csv.DictWriter(fichier, fieldnames=colonnes, delimiter=";")`	Un « écrivain » qui écrit des **dictionnaires** ; `fieldnames` donne l'ordre des colonnes ; `delimiter` le séparateur
`writeheader()`	Écrit la ligne d'en-tête
`writerows(livraisons)`	Écrit une ligne par dictionnaire de la liste (`writerow` pour un seul)

> 🪛 **Essayez** : retirez `newline=""` et relancez sous Windows. Le fichier contient maintenant `\r\r\n` en fin de ligne (deux retours au lieu d'un) : à la relecture, `csv` voit une ligne **vide** après chaque ligne (`[['a', 'b'], [], ['1', '2'], []]`), et Excel affiche une ligne blanche sur deux. Annulez.

5.2 Lire un tableau

Créez `essai7.py` :

```python
import csv

with open("registre.csv", newline="", encoding="utf-8") as fichier:
    lecteur = csv.DictReader(fichier, delimiter=";")
    livraisons = list(lecteur)

print(livraisons[0])
total = sum(int(l["frais"]) for l in livraisons)
print("Total des frais :", total, "F")
```

```
{'date': '2026-10-20', 'quartier': 'Médina', 'poids': '2.5', 'montant': '15000', 'frais': '1000'}
Total des frais : 6000 F
```

Morceau	Ce qu'il veut dire
`csv.DictReader(fichier, delimiter=";")`	Un « lecteur » qui utilise la première ligne comme noms de colonnes, et renvoie chaque ligne comme un **dictionnaire**
`list(lecteur)`	Toutes les lignes, dans une liste
`'poids': '2.5'`	**Attention : tout est du texte.** Un CSV ne connaît pas les types : c'est à vous de convertir avec `int()` ou `float()`
`sum(int(l["frais"]) for l in livraisons)`	La somme des frais convertis en entiers

> 🪛 **Essayez** : retirez `delimiter=";"` du `DictReader`. Le lecteur cherche des **virgules** : chaque ligne devient une seule colonne nommée `date;quartier;poids;montant;frais`, et `l["frais"]` lève une `KeyError: 'frais'`. Annulez.

6. Lire un fichier abîmé sans planter

Les vrais fichiers sont **abîmés** : une virgule à la place d'un point, une colonne oubliée, une valeur impossible. Un bon programme lit ce qu'il peut, **met de côté** ce qu'il ne peut pas lire, et **dit pourquoi**. Créez `abime.csv` :

```
date;quartier;poids;montant;frais
2026-10-20;Médina;2.5;15000;1000
2026-10-20;Pikine;sept;42000;3000
2026-10-21;Yoff;1.2;8000
2026-10-21;Plateau;3.0;-500;1500
2026-10-22;Parcelles;4.5;12000;2000
```

Puis `essai8.py` :

```
import csv


def convertir(ligne):
    poids = float(ligne["poids"])
    montant = int(ligne["montant"])
    frais = int(ligne["frais"])
    if montant <= 0:
        raise ValueError(f"montant négatif ou nul : {montant}")
    return {"date": ligne["date"], "quartier": ligne["quartier"], "poids": poids, "montant":
         montant, "frais": frais}


valides, rejets = [], []
with open("abime.csv", newline="", encoding="utf-8") as fichier:
    for numero, ligne in enumerate(csv.DictReader(fichier, delimiter=";"), start=2):
        try:
            valides.append(convertir(ligne))
        except (ValueError, TypeError) as e:
            rejets.append((numero, str(e)))

print(len(valides), "lignes valides,", len(rejets), "rejetées")
for numero, raison in rejets:
    print(f"  ligne {numero} : {raison}")
```

```
2 lignes valides, 3 rejetées
  ligne 3 : could not convert string to float: 'sept'
  ligne 4 : int() argument must be a string, a bytes-like object or a real number, not 'NoneType'
  ligne 5 : montant négatif ou nul : -500
```

Morceau	Ce qu'il veut dire
`convertir(ligne)`	Une fonction qui transforme une ligne de textes en vraie livraison, ou **lève** une exception (chapitre 1, section 5)
`enumerate(..., start=2)`	Le numéro de ligne **dans le fichier** : la ligne 1 est l'en-tête, la première livraison est donc la ligne 2. C'est ce numéro qu'il faut donner à celui qui corrigera le fichier
`except (ValueError, TypeError) as e:`	Les deux types d'erreurs possibles ici, entre parenthèses
`rejets.append((numero, str(e)))`	On garde le numéro et la raison, pour le bilan

La ligne 4 n'a que 4 valeurs : `DictReader` met `None` dans la colonne manquante (`frais`), et `int(None)` lève une `TypeError` . La ligne 5 est lisible, mais **absurde** : c'est notre propre `raise` qui la refuse.

> 🔵 **Standard : où mettre la frontière ?** Le programme doit-il refuser une ligne, ou la corriger (remplacer `2,5` par `2.5`) ? Corriger en silence est dangereux : on peut se tromper sur l'intention. Une bonne pratique : corriger seulement ce qui est **certain** (les espaces, la virgule décimale), et refuser le reste en le **signalant**. Dans tous les cas, le bilan dit combien de lignes ont été lues, corrigées, rejetées.

7. Le format JSON

7.1 Enregistrer un dictionnaire

Le **JSON** (*JavaScript Object Notation*) ressemble beaucoup aux dictionnaires et aux listes de Python. On l'utilise pour les configurations et pour échanger des données avec les sites web. Créez `essai9.py` :

```
import json

tarifs = {"Médina": 1000, "Plateau": 1500, "Yoff": 2000, "Parcelles": 2000, "Pikine": 2500}
with open("tarifs.json", "w", encoding="utf-8") as fichier:
    json.dump(tarifs, fichier, indent=2, ensure_ascii=False)

with open("tarifs.json", encoding="utf-8") as fichier:
    relu = json.load(fichier)
print(relu == tarifs, relu["Pikine"])
with open("tarifs.json", encoding="utf-8") as fichier:
    print(fichier.read())
```

```
True 2500
{
  "Médina": 1000,
  "Plateau": 1500,
  "Yoff": 2000,
  "Parcelles": 2000,
  "Pikine": 2500
}
```

Morceau	Ce qu'il veut dire
`json.dump(objet, fichier)`	Écrit l'objet (dictionnaire, liste, textes, nombres, `True` / `False` / `None`) dans le fichier
`indent=2`	Présente le fichier sur plusieurs lignes, avec 2 espaces de décalage : lisible par un humain
`ensure_ascii=False`	Garde les accents tels quels (sinon « Médina » devient `"M\u00e9dina"`)
`json.load(fichier)`	Relit le fichier et reconstruit le dictionnaire
`relu == tarifs`	`True` : on retrouve **exactement** les mêmes données, **avec leurs types** (2500 est un nombre, pas un texte, contrairement au CSV)

`json.dumps(objet)` et `json.loads(texte)` (avec un **s**, pour *string*) font la même chose avec un texte au lieu d'un fichier.

7.2 Ce que JSON ne sait pas écrire

Créez `essai10.py` :

```python
import json
from datetime import date

livraison = {"date": date(2026, 10, 23), "quartier": "Yoff", "frais": 2000}
try:
    print(json.dumps(livraison))
except TypeError as e:
    print("Erreur :", e)
livraison["date"] = livraison["date"].isoformat()
print(json.dumps(livraison, ensure_ascii=False))
print(date.fromisoformat("2026-10-23"))
```

```
Erreur : Object of type date is not JSON serializable
{"date": "2026-10-23", "quartier": "Yoff", "frais": 2000}
2026-10-23
```

JSON ne connaît que les textes, les nombres, les booléens, `null`, les listes et les dictionnaires. Une **date** doit être transformée en texte avant l'écriture (`isoformat()` donne `"2026-10-23"`), et retransformée en date à la lecture (`date.fromisoformat(...)`).

Python	JSON
`dict`	objet `{...}` (les clés deviennent **toujours** des textes)
`list`, `tuple`	tableau `[...]` (un tuple revient sous forme de **liste**)
`str`, `int`, `float`	texte, nombre
`True`, `False`, `None`	`true`, `false`, `null`
`date`, `set`, objets...	**impossible** directement : à convertir

8. Les erreurs de fichiers

Créez `essai11.py` :

```
import json


def charger_tarifs(nom):
    try:
        with open(nom, encoding="utf-8") as fichier:
            return json.load(fichier)
    except FileNotFoundError:
        print(f"{nom} introuvable : tarifs par défaut")
    except json.JSONDecodeError as e:
        print(f"{nom} abîmé ({e}) : tarifs par défaut")
    return {"Médina": 1000}


with open("casse.json", "w", encoding="utf-8") as f:
    f.write('{"Médina": 1000, "Yoff": }')
print(charger_tarifs("tarifs.json"))
print(charger_tarifs("absent.json"))
print(charger_tarifs("casse.json"))
```

```
{'Médina': 1000, 'Plateau': 1500, 'Yoff': 2000, 'Parcelles': 2000, 'Pikine': 2500}
absent.json introuvable : tarifs par défaut
{'Médina': 1000}
casse.json abîmé (Expecting value: line 1 column 26 (char 25)) : tarifs par défaut
{'Médina': 1000}
```

`JSONDecodeError` indique où le fichier est abîmé : ligne 1, colonne 26, là où une valeur était attendue après `"Yoff":`. C'est une sous-classe de `ValueError` (chapitre 1, section 6).

Exception	Quand	Exemple de message
`FileNotFoundError`	Le fichier n'existe pas (ou le chemin est faux)	`[Errno 2] No such file or directory: 'registre_absent.csv'`

Exception	Quand	Exemple de message
`PermissionError`	Pas le droit, fichier ouvert dans Excel sous Windows, ou un **dossier** ouvert comme un fichier	`[Errno 13] Permission denied: 'registres'`
`FileExistsError`	Mode `"x"` sur un fichier existant	
`UnicodeDecodeError`	Mauvais encodage (section 3)	`'utf-8' codec can't decode byte 0xe0...`
`json.JSONDecodeError`	Fichier JSON mal formé	`Expecting value: line 1 column 26 (char 25)`

Toutes, sauf les deux dernières, sont des sous-classes de `OSError` : `except OSError` les attrape ensemble.

Exercices

Créez un dossier `exercices-ch2`. Pour chaque exercice, préparez d'abord les fichiers indiqués.

- ★ **Exercice 1 : compter.** Créez `proverbes.txt` avec ces trois lignes : `Ndank ndank mooy japp golo ci ñaay.`, `Petit à petit, on attrape le singe dans la forêt.`, `Ce proverbe wolof enseigne la patience.` Écrivez `statistiques(chemin)` qui renvoie le tuple (nombre de lignes, nombre de mots, nombre de caractères). Résultat attendu : `(3, 23, 126)`. Que se passe-t-il avec un fichier qui n'existe pas ? (Sections 2 et 4.)
- ★ **Exercice 2 : le carnet de contacts.** Enregistrez dans `contacts.csv` (séparateur `;`) trois contacts : Ndeye Diop, 77 123 45 67, Thiès ; Babacar Fall, 76 987 65 43, Dakar ; Aïssatou Ba, 78 555 44 33, Thiès. Relisez le fichier et affichez seulement les contacts de Thiès. (Section 5.)
- ★★ **Exercice 3 : les notes abîmées.** Créez `notes.csv` avec les colonnes `etudiant;matiere;note` et ces lignes : Awa, Algorithmique, 14 ; Moussa, Algorithmique, 12,5 ; Awa, Bases de données, 16 ; Moussa, Bases de données, abs ; Khady, Algorithmique, 18 ; Khady, Bases de données, 23 ; Moussa, Réseaux, 11. Calculez la moyenne de chaque étudiant en acceptant la virgule décimale, et en refusant les notes illisibles ou hors de 0 à 20, avec leur numéro de ligne. Attendu : Awa 15,00 ; Moussa 11,75 ; Khady 18,00 ; deux rejets (lignes 5 et 7). (Section 6.)
- ★★ **Exercice 4 : la configuration.** Écrivez `charger_config(chemin)` : si le fichier JSON n'existe pas, il est **créé** avec la configuration par défaut `{"langue": "fr", "taille_police": 12, "theme": "clair"}` ; s'il existe, ses valeurs **remplacent** celles par défaut (indice : `{**defaut, **lu}`) ; s'il est abîmé, on garde la configuration par défaut. Testez les trois cas. (Sections 4, 7 et 8.)
- ★★★ **Exercice 5 : fusionner les boutiques.** Dans un dossier `ventes`, créez trois fichiers CSV (`produit;quantite;prix`) : `boutique_medina.csv` (Savon 10 x 500, Sucre 4 x 800), `boutique_yoff.csv` (Riz 3 x 12 500, Huile 2 x 1 750), `boutique_pikine.csv` (Savon 6 x 500). Écrivez un programme qui fusionne **tous** les fichiers du dossier (sans connaître leurs noms à l'avance) dans `toutes_ventes.csv`, avec une colonne `source` (le nom du fichier sans extension), et affiche le chiffre d'affaires total. Attendu : 5 lignes depuis 3 fichiers, 52 200 F. (Sections 4 et 5.)

Atelier 2 : enregistrer et recharger le registre

Le travail principal du chapitre : vous chargez le registre des livraisons d'octobre de Yonnee Express, un fichier CSV réel de l'entreprise (imaginaire) avec ses erreurs de saisie ; vous validez chaque ligne, vous écrivez le fichier des rejets avec leurs raisons, puis vous réenregistrez le registre propre en CSV et en JSON, avec des tarifs lus dans un fichier de configuration. L'énoncé et le fichier du registre vous sont fournis séparément.

A retenir

- `with open(chemin, mode, encoding="utf-8") as fichier:` ouvre et **referme toujours** le fichier. Modes : `"r"` lire, `"w"` écrire en effaçant, `"a"` ajouter, `"x"` créer.
- **Toujours** `encoding="utf-8"` ; une `UnicodeDecodeError` sur un vieux fichier : essayer `"cp1252"`.
- `pathlib.Path` : `dossier / "nom.csv"`, `mkdir(exist_ok=True)`, `exists()`, `glob("*.csv")`, `read_text` / `write_text`.
- CSV : `csv.DictWriter` / `csv.DictReader`, avec `newline=""` et `delimiter=";"`. **Tout est lu comme du texte** : convertir soi-même.
- Fichier abîmé : une fonction `convertir` qui lève une exception, une boucle avec `try` / `except`, une liste de rejets avec le **numéro de ligne** et la **raison**.
- JSON : `json.dump` / `json.load`, `indent=2`, `ensure_ascii=False` ; les types sont conservés, sauf les dates (`isoformat` / `fromisoformat`).
- Erreurs de fichiers : `FileNotFoundError`, `PermissionError` (des `OSError`), `UnicodeDecodeError`, `json.JSONDecodeError`.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`FileNotFoundError: [Errno 2] No such file or directory: '...'`	Fichier absent, nom mal écrit, ou programme lancé depuis un autre dossier	Vérifier le nom ; lancer depuis le bon dossier ; `Path(...).exists()`
`PermissionError: [Errno 13] Permission denied: '...'`	Fichier ouvert dans Excel (Windows), pas de droits, ou chemin d'un dossier	Fermer le fichier dans Excel ; vérifier le chemin
`UnicodeDecodeError: 'utf-8' codec can't decode byte ...`	Fichier écrit dans un autre encodage	`encoding="cp1252"` à la lecture
Les accents s'affichent `Ã©`	Fichier UTF-8 lu en cp1252 (l'inverse du cas précédent)	`encoding="utf-8"`
Le fichier est vide ou a perdu son contenu	Ouvert en mode `"w"` au lieu de `"a"`	Utiliser `"a"` pour ajouter
Une ligne vide sur deux dans le CSV (Windows)	`newline=""` oublié à l'écriture	Ajouter `newline=""`

Message ou symptôme	Ce que ça veut dire	Solution
`KeyError: 'frais'` sur une ligne de CSV	Mauvais séparateur : toute la ligne est dans une seule colonne	`delimiter=";"`
`TypeError: int() argument must be ... not 'NoneType'`	Une colonne manque sur cette ligne du CSV	Attraper l'erreur et rejeter la ligne
`TypeError: Object of type date is not JSON serializable`	Une date (ou un `set`) dans les données JSON	`.isoformat()` avant `json.dump`
`json.JSONDecodeError: Expecting value: line ... column ...`	Fichier JSON mal formé	Lire la position indiquée ; virgule ou guillemet en trop

Pour aller plus loin

- Le tutoriel officiel, « Lecture et écriture de fichiers » (en français) : https://docs.python.org/fr/3/tutorial/inputoutput.html#reading-and-writing-files
- Le module `csv` : https://docs.python.org/fr/3/library/csv.html
- Le module `pathlib` : https://docs.python.org/fr/3/library/pathlib.html
- Le module `json` : https://docs.python.org/fr/3/library/json.html